Lingo Bharat - Hackathon Submission

Project Information

Field	Value
Project Name	Lingo Bharat
Tagline	The first localization QA tool that actually understands India
Author	Kshitij
Hackathon	Lingo.dev Hackathon 2026 (Week 2)
Repository	https://github.com/AjayPoshak/Lingo_Bharat
Built With	TypeScript, Node.js, Lingo.dev SDK, React, Vite

---

1. What It Does

Lingo Bharat is an open-source CLI tool that acts as an "ESLint for Indic Localization." It analyzes translation files, CSS, and source code to detect India-specific localization failures that generic i18n tools completely miss.

It produces a **Bharat Readiness Score (BRS)** from 0 to 100, graded A through F, across 5 checks:

Check	What It Catches	Severity
Number Format	International comma grouping (1,234,567) instead of Indian Lakh/Crore (12,34,567)	Critical
Font Stack	Missing Indic script font fallbacks in CSS (causes tofu/empty boxes)	Critical
Formality Register	Inappropriate tu-register pronouns in Hindi (offensive in formal contexts)	High
String Overflow	Indic translations exceeding container width due to script expansion	Medium
Coverage Reach	Percentage of India's 900M internet users reached by supported locales	Strategic

---

2. How Lingo.dev Is Used

This project deeply integrates with the Lingo.dev ecosystem at every layer:

Integration Point 1: `i18n.json` Config Compatibility

Lingo Bharat reads the exact same `i18n.json` configuration file used by the Lingo.dev CLI. It parses buckets, locale targets, ignoredKeys, lockedKeys - all from the same config. Zero additional setup for Lingo.dev users.

File: `packages/core/src/parsers/i18n-config.ts`

Integration Point 2: `LingoDotDevEngine.recognizeLocale()`

During the analysis pipeline, Lingo Bharat samples translation values from each locale file and calls `recognizeLocale()` to validate that the file actually contains the declared language. This catches stale, misplaced, or corrupted translation files.

File: `packages/core/src/parsers/i18n-config.ts` (lines 128-173)

```
const engine = new LingoDotDevEngine({ apiKey });
const detected = await engine.recognizeLocale(sampleText);
```

Integration Point 3: `LingoDotDevEngine.localizeObject()`

The `FormalityFixer` class uses `localizeObject()` to batch re-translate all formality violations in a single API call. Instead of calling `localizeText()` per-string (N API calls), it groups all violating keys into one object and translates them together (1 API call).

File: `packages/core/src/fixer/formality-fixer.ts` (lines 48-61)

```
const engine = new LingoDotDevEngine({ apiKey: this.apiKey });
const translated = await engine.localizeObject(sourceObject, {
  sourceLocale,
  targetLocale,
});
```

Integration Point 4: `LingoDotDevEngine.localizeText()`

For individual string re-translation fallback scenarios, Lingo Bharat uses `localizeText()`.

File: `packages/cli/src/commands/fix.ts`

Integration Point 5: CI/CD Action with `lingodotdev/lingo.dev@main`

The GitHub Action implements a quality gate pipeline:

1. `lingodotdev/lingo.dev@main` translates files
2. `lingo-bharat check` validates translation quality
3. Posts BRS score as a PR comment
4. Uploads report artifact

File: `.github/workflows/lingo-bharat.yml`

---

3. Technical Architecture

```
              i18n.json (Lingo.dev config)
                        |
                        v
            +---------------------+
            | buildCheckerContext  |
            |  - parseLingoConfig |
            |  - resolveFiles     |
```

```
          |  - recognizeLocale  | <-- Lingo.dev SDK
          +---------------------+
                      |
       +------------+------------+
       v            v            v
  Number Format  Font Stack  Formality Register
       v            v
  String Overflow  Coverage Reach
       |            |
       +-----+------+
             |
             v
     +----------------+
     | Scoring Engine |
     |  BRS 0-100     |
     |  Grade A-F     |
     +----------------+
             |
     +--------+--------+
     v        v        v
  Terminal  JSON     Dashboard
  Report   Report    (React)
```

Monorepo Structure

Package	Description	Key Files
`packages/core`	Analysis engine, checks, scoring, fixer	15 source files
`packages/cli`	Commander.js CLI (check, fix, serve, report)	7 source files
`packages/dashboard`	Vite + React visual dashboard	SPA
`apps/demo`	PayBharat demo app with seeded bugs	Next.js

Test Coverage

Test File	Tests	What It Covers
`engine.test.ts`	4	BRS calculation, grades, coverage math
`number-format.test.ts`	1	International vs Indian format detection
`font-stack.test.ts`	4	Missing fonts, Devanagari pass, multi-locale, autoFix
`formality-register.test.ts`	6	Tu-register pronouns, verbs, aap pass, non-Hindi skip
`string-overflow.test.ts`	5	Tamil overflow, Hindi pass, empty keys, severity
`coverage.test.ts`	7	Single/multi locale, recommendations, effort levels
`integration.test.ts`	5	Full pipeline, weighted BRS, grade boundaries
Total	**32 tests**	

---

4. Demo Flow

Before Fix (BRS: ~24, Grade F)

```
cd apps/demo
pnpm reset                                    # Restore seeded bugs
npx lingo-bharat check --config ./i18n.json   # See violations
```

Auto-Fix

```
npx lingo-bharat fix --config ./i18n.json     # SDK batch re-translation
```

After Fix (BRS: ~85, Grade B)

```
npx lingo-bharat check --config ./i18n.json   # Verify improvements
```

Dashboard

```
npx lingo-bharat serve --config ./i18n.json --open
```

---

5. What Makes This Unique

1. **India-specific**: No other tool checks for Lakh/Crore number formatting, tu/aap formality register, or Indic script font stack requirements.
2. **Complementary to Lingo.dev**: It's not a competing translation tool. It's a quality gate that runs *after* translation. Lingo.dev makes your translations, Lingo Bharat makes sure they're actually ready for India.
3. **Actionable**: Every violation has a severity, explanation, and most have auto-fix. The fixer uses the Lingo.dev SDK to batch re-translate violations.
4. **Coverage-aware**: It doesn't just check what you have - it tells you what's missing. With population-weighted recommendations, it shows exactly which locales to add for maximum reach.
5. **Self-dogfooding**: The project itself has an `i18n.json` at the root with 5 Indic target locales, ready for Lingo.dev to translate its own CLI messages.

---

6. Setup Instructions

Prerequisites

- Node.js 20+
- pnpm 9+
- Lingo.dev API key (for SDK features)

Install & Build

```
git clone https://github.com/AjayPoshak/Lingo_Bharat.git
cd Lingo_Bharat
pnpm install
pnpm build
```

Set API Key

```
export LINGODOTDEV_API_KEY="lingo_sk_..."
```

Run Tests

```
pnpm test   # 32 tests, 7 files, all passing
```

Run Demo

```
cd apps/demo
pnpm reset
npx lingo-bharat check --config ./i18n.json --verbose
npx lingo-bharat fix --config ./i18n.json
npx lingo-bharat check --config ./i18n.json
npx lingo-bharat serve --config ./i18n.json --open
```

---

7. Files Changed / Created

New Files (10)

File	Purpose
`packages/core/src/fixer/formality-fixer.ts`	FormalityFixer class using localizeObject()
`packages/core/src/fixer/index.ts`	Fixer module exports
`packages/core/test/font-stack.test.ts`	Font stack check tests (4 tests)
`packages/core/test/formality-register.test.ts`	Formality register tests (6 tests)
`packages/core/test/string-overflow.test.ts`	String overflow tests (5 tests)
`packages/core/test/coverage.test.ts`	Coverage check tests (7 tests)
`packages/core/test/integration.test.ts`	Integration tests (5 tests)
`i18n.json`	Root config for self-dogfooding
`locales/en.json`	English CLI messages for self-dogfooding
`docs/submission/`	This submission + video script

Modified Files (10)

File	Change
`packages/core/src/types.ts`	Added LocaleValidationResult, localeValidation to CheckerContext & BharatReport
`packages/core/src/parsers/i18n-config.ts`	Added validateLocales() using recognizeLocale()
`packages/core/src/index.ts`	Exported FormalityFixer, LocaleValidationResult
`packages/core/src/scoring/engine.ts`	buildReport() accepts localeValidation param
`packages/cli/src/index.ts`	Added --report-path option
`packages/cli/src/commands/check.ts`	Added report writing via --report-path
`packages/cli/src/commands/fix.ts`	Refactored to batch FormalityFixer
`packages/cli/src/commands/report.ts`	Pass localeValidation to buildReport
`packages/cli/src/renderers/terminal.ts`	Display locale validation results
`packages/cli/package.json`	Moved lingo.dev to dependencies
`.github/workflows/lingo-bharat.yml`	Full 5-step CI pipeline
`README.md`	Integration showcase, architecture, badges

---

8. Lingo.dev SDK Methods Used (Summary)

SDK Method	Where Used	Purpose
`recognizeLocale()`	`i18n-config.ts`	Validate translation file languages
`localizeObject()`	`formality-fixer.ts`	Batch re-translate formality violations
`localizeText()`	`fix.ts` (fallback)	Individual string re-translation
`i18n.json` config	`i18n-config.ts`	Read same config as Lingo.dev CLI
`lingodotdev/lingo.dev@main`	GitHub Action	CI translate step before quality gate